Lecture 3: Improving Performance

From a Working Prototype to Reliable Results

University of Chicago

April 7, 2026

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

Review: What We've Covered

Lecture 1: Foundations

- Basic AI terminology (LLMs, tokens, context windows, APIs)
- How to work with these models programmatically
- Understanding pricing and token economics

Lecture 2: Building AI Systems

- Vertical slices – prove end-to-end flow works first
- Crawl, Walk, Run – start simple, add complexity incrementally
- Built a resume scorer: resume + job req $\rightarrow$ 0–100 score

Where We're Going

Last time: We built a working resume scorer

- Single prompt, single call, returns a score
- It works! But how *well* does it work?

Today: Techniques to improve AI system performance

- Walk through common failure modes
- Learn a toolkit of solutions
- Apply them to the resume scorer in the notebook

Framework: Problem → Why it happens → Solution

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

Our Starting Point

The resume scorer from Lecture 2:

$$\boxed{\text{Resume Text}} + \boxed{\text{Job Requirements}} \longrightarrow \boxed{\text{LLM: “Score 0–100”}} \longrightarrow \boxed{\text{Score: 72}}$$

This is a vertical slice – it works end-to-end.
Now: how do we make it *good*?

What Does “Good” Look Like?

- What are some characteristics of good?
- (Open Question)

What Does "Good" Look Like?

- **Accurate** – scores reflect actual candidate quality
- **Consistent** – same resume gets the same score every time
- **Differentiated** – strong candidates score higher than weak ones
- **Explainable** – you can justify why a candidate got their score

Model Iteration

Best practice: Create a measurable result, then iterate

1. **Label some data** – identify resumes where you know the answer
2. **Run your prompt** – see how the model performs
3. **Identify failures** – where does it get things wrong?
4. **Adjust and re-run** – change the prompt, check if results improve

This is the loop we'll follow today: identify a problem, apply a fix, measure improvement.

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

Problem: Everything Gets a Similar Score

You run the scorer on 6 resumes and get:

Resume	Score
Resume A	74
Resume B	71
Resume C	68
Resume D	72
Resume E	66
Resume F	70

Everything is between 66–74. The scorer can't tell the difference between a great candidate and a poor one.

Why Score Bunching Happens

LLMs default to "safe" middle-range answers:

- Training data is full of moderate, hedging language
- Without strong guidance, models avoid extremes
- A vague prompt like "score 0–100" gives the model no anchor points

The model doesn't know what you mean by "good":

- Is 5 years of Python experience good or average?
- Is a Master's degree required or a nice-to-have?
- Should missing a skill be a small or large penalty?

Without criteria, the model invents its own – and they're inconsistent.

Solution: Domain Context and Grounding

Grounding = anchoring the model's output in specific facts, criteria, or source material rather than letting it generate from general patterns.

Tell the model exactly what matters and how much:

Instead of: "Score this resume 0–100"

Provide:

- **Specific criteria** with weights or importance levels
- **What "high" and "low" mean** for this role
- **Must-haves vs. nice-to-haves**
- **Anchor points**: "A score of 90+ means the candidate meets all requirements. Below 40 means missing critical skills."

The more specific your criteria, the more differentiated the scores

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

Problem: The Model Makes Things Up

Hallucination = when the model generates confident, plausible-sounding content that is not supported by the input data.

You ask the model to explain its score:

> *"The candidate has 8 years of Python experience and demonstrated strong leadership at a Fortune 500 company..."*

But the resume says: 3 years of Python, worked at a 20-person startup.

The model generated a plausible-sounding justification that doesn't match the actual resume.

Why Hallucination Happens

LLMs are pattern completers, not fact checkers:

- They generate text that *sounds right* based on patterns
- "Senior developer" in training data often correlates with "8+ years"
- Without explicit instruction to reference the source, they fill gaps

This is especially dangerous for scoring:

- Score might be based on hallucinated qualifications
- You can't trust the reasoning to explain the score
- Impossible to audit or defend decisions

Solution: Require Evidence (Grounding)

Force the model to cite its sources:

- Require **exact quotes** from the resume for every claim
- Ask for evidence *before* the conclusion
- Include an "evidence" field in structured output
- Validate that cited text actually appears in the resume

Result: Instead of inventing "8 years of Python," the model must point to the actual text: "Python developer, 2021–2024"

Grounding turns the LLM from a "creative writer" into an "evidence-based analyst"

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

Problem: Same Resume, Different Scores

Run the same resume through the scorer three times:

Run	Score
Run 1	78
Run 2	65
Run 3	82

A 17-point spread on the same input.

- Is this candidate a 65 or an 82?
- Would you trust this system to rank candidates?
- What if the hiring decision depends on a threshold?

Why Inconsistency Happens

LLMs are non-deterministic by design:

- Temperature controls randomness in token selection
- Even at low temperature, outputs can vary
- Vague instructions amplify variance – many "correct" answers

The less constrained the task, the more variance you get:

- "Score 0–100" has enormous output space
- "Classify as: Strong Match / Partial Match / No Match" has 3 options
- More structure = less room for random drift

Solution: Temperature and Structured Output

Reduce randomness and constrain the output:

- **Lower temperature** (0.0–0.3) for classification and scoring tasks
  - Less creative, more consistent
  - Save high temperature for brainstorming, not evaluation
- **Structured output schemas** force specific fields and types
  - JSON schema with required fields
  - Constrained ranges (score must be 0–100)
  - Enumerated categories instead of free text
- **Specific criteria** reduce subjective interpretation
  - "Does the candidate have 5+ years Python?" has less variance than "Rate their Python skills"

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

Problem: The Scorer Ignores Important Factors

A strong candidate gets a mediocre score because:

- The scorer focused on keyword matching
- Missed that the candidate led a relevant project
- Didn't weigh industry experience appropriately
- Treated a PhD the same as a Bachelor's

A weaker candidate scores nearly as high because:

- Had the right keywords sprinkled throughout
- Model couldn't distinguish depth from breadth
- Single prompt tries to evaluate everything at once

One prompt asking "score this resume" is too much at once.

Solution: Decomposition

Break the scoring into focused sub-tasks:

Resume $\longrightarrow$ Extract Experience / Extract Skills / Extract Education $\longrightarrow$ Combine & Score (in code)

Each sub-task is simpler and more focused:

- Easier for the model to get right
- Easier for you to debug (which step went wrong?)
- Can use different prompts/models for different steps

Decomposition: Key Idea

Use LLMs for what they're good at:

- Extracting information from unstructured text
- Classification and categorization
- Summarization

Use code for what code is good at:

- Conditional logic (if/then rules)
- Math and scoring formulas
- Combining results from multiple steps
- Enforcing business rules consistently

LLMs extract and classify. Code decides and calculates.

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

Problem: Ambiguous Resumes Get Random Scores

Some resumes are genuinely hard to evaluate:

- Career changer with transferable but non-obvious skills
- Experience in adjacent technology (Java listed, role needs Python)
- Years of experience in a broader category but not the specific area

Without guidance, the model handles these inconsistently:

- Sometimes gives benefit of the doubt, sometimes doesn't
- No consistent policy for "close but not exact" matches
- Each run may interpret the ambiguity differently

Solution: Few-Shot Examples

Show the model how you want edge cases handled:

Include 2–5 examples in your prompt that cover:

1. **A clear strong match** – what does "high score" look like?
2. **A clear weak match** – what earns a low score?
3. **An ambiguous case** – how should the model handle it?

Examples are more powerful than instructions:

- "Score based on relevant experience" is vague
- Showing a scored example of "5 years Java → 60 for a Python role" is concrete
- The model learns your *policy*, not just your words

Few-shot examples encode your judgment for edge cases

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

Changing Models and Role Specification

Not all models are equal for every task:

- **Larger models** (GPT-4, Claude Opus) – better reasoning, more expensive
- **Smaller models** (Claude Haiku, GPT-4o-mini) – faster, cheaper, good for simple tasks
- Start with a capable model, then try cheaper ones and see if quality holds
- Sometimes switching models fixes problems that prompt engineering can't

Role/persona specification – set the model's "mindset":

- "You are a hiring manager at a tech company" $\rightarrow$ evaluates like a practitioner
- "You are a senior engineer reviewing technical qualifications" $\rightarrow$ focuses on depth
- Activates relevant patterns from training data; one sentence is usually enough

Chain-of-Thought and Output Validation

Chain-of-thought – ask the model to reason before answering:

- "First list the relevant qualifications, then evaluate each one"
- Forces organized reasoning before committing to an answer
- Tradeoff: more tokens (cost and latency) for better accuracy

Output validation and guardrails – don't trust, verify:

- **Schema validation**: Use structured output (e.g., Pydantic) to enforce field types and ranges
- **Range checks**: Score must be 0–100, not 150 or −5
- **Re-prompting**: If output fails validation, try again (with a limit)
- Combine AI flexibility with code reliability – a safety net, not a replacement for good prompts

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

When Prompting Isn't Enough: Fine-Tuning

Fine-tuning = training a model on your specific data

Consider fine-tuning when:

- You have *lots* of labeled examples
- Prompt engineering has plateaued and you need more consistency
- You need to reduce cost (fine-tuned small models can replace large ones)
- You have a very domain-specific task with specialized vocabulary

Don't fine-tune when:

- You haven't tried prompt engineering thoroughly first
- Your labeled data is small or noisy
- Requirements change frequently (re-training is expensive)

Rule of thumb: Exhaust prompt engineering before fine-tuning.

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

The Improvement Toolkit

Problem	Solution
Scores bunch together	Domain context, anchor points
Model makes things up	Grounding with citations
Results vary across runs	Temperature, structured output
Misses important factors	Decomposition
Edge cases handled randomly	Few-shot examples
Wrong "personality" for task	Role/persona specification
Reasoning is shallow	Chain-of-thought
Bad outputs slip through	Output validation
Prompt engineering plateaus	Change model or fine-tune

Order of Operations

When improving an AI system, try techniques in this order:

1. **Get specific** – Add domain context, criteria, and anchor points
2. **Add structure** – Structured output, lower temperature
3. **Ground it** – Require citations and evidence
4. **Show examples** – Few-shot prompting for edge cases
5. **Break it apart** – Decompose into sub-tasks
6. **Try a different model** – Bigger, smaller, or specialized
7. **Fine-tune** – Only after exhausting the above

Each step is more effort than the last. Start cheap and simple.

Outline

1. Where We Are
2. The Single-Shot Baseline
3. Common Problems and Solutions
4. Problem 1: Score Averaging
5. Problem 2: Hallucinated Reasoning
6. Problem 3: Inconsistency
7. Problem 4: Missing Nuance
8. Problem 5: Edge Cases
9. More Techniques
10. Beyond Prompt Engineering
11. Putting It All Together
12. Your Turn

Today's Exercise

Open the notebook: `lecture_3_resume_scorer_improvement.ipynb`

You have 6 resumes:

- 2 **gold** – strong matches (should score high)
- 2 **silver** – decent matches (should score moderately)
- 2 **wild** – unknown quality (likely score lower)

Your goal:

- Start with the baseline single-shot scorer
- Apply the techniques from today's lecture
- Improve separation: gold $>$ silver $>$ wild
- Submit your scores to the leaderboard

What to Try

Pick techniques and experiment:

- Rewrite the prompt with specific scoring criteria
- Add a role/persona to the system message
- Include few-shot examples of good and bad resumes
- Decompose into extraction steps and combine in code
- Require evidence citations for the score justification
- Try different models and compare results
- Adjust temperature for more consistent outputs

Measure your progress: Do gold resumes reliably outscore silver? Do silver outscore wild?

Let's Get Started!

Open the notebook:

`lecture_3_resume_scorer_improvement.ipynb`

Apply the techniques

Improve gold vs. silver vs. wild separation

Submit to the leaderboard